**Cloud Computing**

Exercise - 6 Total points: 10

---

1. Calculate the Monte Carlo PI by using the RMI approach. Let the client initiate the call with number of threads and random numbers to be considered per thread as parameters. Let the server utilize thread programming to calculate the PI value and return it to the client. (2 points)

2. In this task, we will build a Book management application and create a REST API for it. The API will include methods to view books downloaded from Gutenberg, download new books, view the content of a book, delete books and search whether specific words occur in a book.

   We will use the Python Flask framework to implement the HTTP API server component. The Flask framework provides a simple programming interface for creating HTTP APIs and web services and supports all the common HTTP API features.

   **Software used and configuration**

   You can choose the Python programming environment that suits you, but it is recommended to use the PyCharm Python IDE because it simplifies project management, and setting up virtual environments. The recommended Python version is Python 3.x, as the solutions to these tasks have been tested in versions 3.9 and 3.13.

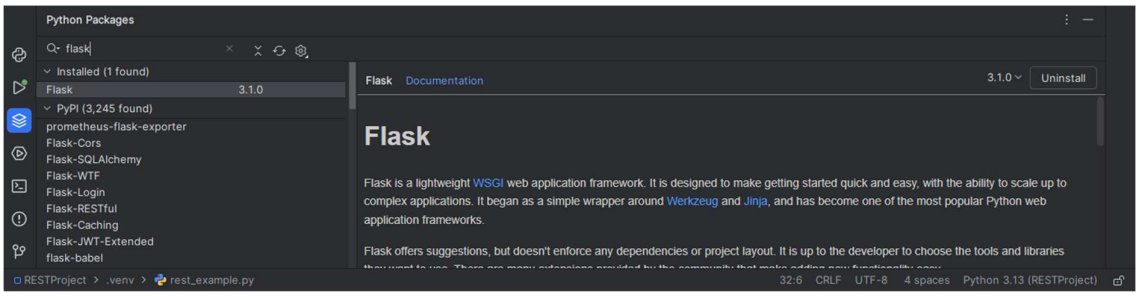
   We will set up everything needed to start developing an HTTP API in Python. We will also download and investigate an example HTTP API implementation that we will modify and extend in the following exercises.

   Create a new Python PyCharm project and choose Python 3.9 when you are asked which Python interpreter to use.

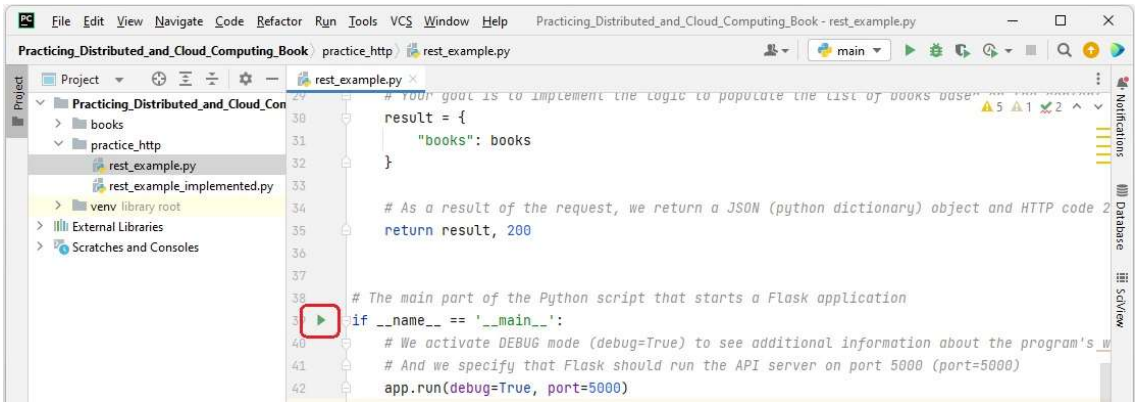
   Download the example Flask HTTP API program named *rest_example.py* (provided in moodle), move it into the Pycharm project and familiarize yourself with the code.

   The example application currently only contains a single HTTP API endpoint */books* and a single GET method implementation *list_of_books()*. At the moment, the code will just return an empty list, when a GET request is sent to the */books* API endpoint. We will properly implement the logic later. Right now, use this example program to test whether you can successfully run the Flask application server and send a GET request to it.

   This example code requires a few Python libraries to be installed before we can run it. Install the latest *requests* and *flask* Python packages inside the Pycharm project:




Let's run the example program and check that it works correctly. Click the green triangle "play" button next to the line containing if __name__ == '__main__':

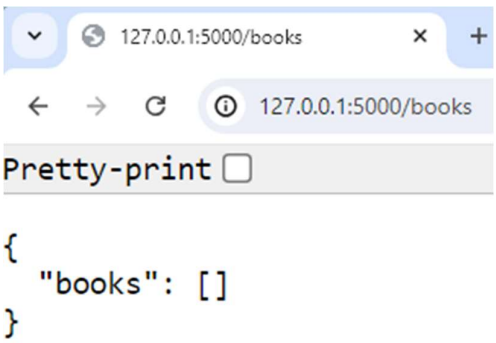



This will start the Flask web server in the local machine and we can call the API methods either directly through the browser (GET methods) or from the command line (GET, DELETE, POST methods).

Let's check that the API works correctly. We will send a GET request to the HTTP API */books* endpoint. In this example, the Flask server is configured to run on port 5000 and we can access the server from the localhost address.

You can first try accessing the server address directly from the browser. Open the page http://localhost:5000/books in any browser. Browser will send a GET HTTP request and display the result of the request.

The expected result is:



Next, let's try sending a GET request through the command line.

If you are using **Windows**, open the PowerShell application and use the following PowerShell command:

```
Invoke-RestMethod -Uri "http://localhost:5000/books" -Method 'Get'
```

If you are using **Linux**, open command line terminal and use the following curl command:

```
curl http://localhost:5000/books
```

The expected result is the same, as you saw with the browser.

### a. Creating an HTTP method for listing books

In this task, we will modify the example application's HTTP GET method so that it returns a list of books managed by the application, as a JSON data structure. We (initially) manually prepare a few books for testing the new functionality.

Create a folder named books inside the Python project folder. This is where our program will store the book files.

In this exercise, your goal is to modify the list_of_books() method inside the rest_example.py file and make it properly return the list of books, instead of an empty list.

This method is executed when a GET HTTP method is requested in our application */books* API endpoint address. In the code, this is controlled by the Flask @app.route() decorator statement before the method definition:

```
@app.route('/books', methods=['GET'])
def list_of_books():
```

Modify the content of the list_of_books() method, and add code that generates a Python list that contains all the names of the files that exist in the books folder.
You can use the Python method listdir(folder_name) to get a list of files, like this:

```
files_in_folder = os.listdir(books_folder)
```

Then, for each file that exists in the books folder, append the name of the file into the books list object named books:

```
for file in os.listdir(book_folder):
    if file.endswith(".txt"):
        books.append(file_name.split(".")[0][2:])
```

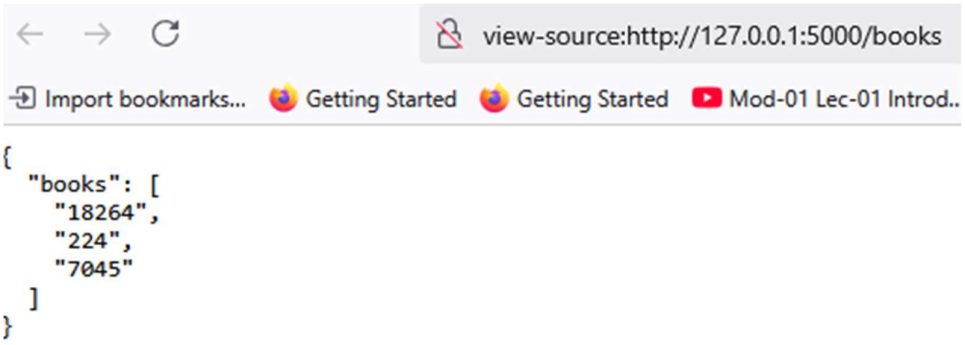
This code also checks that the file ends with “.txt” and removes the ".txt" suffix and the “pg” prefix from the file name, before adding the resource name to the list.

The list_of_books() method returns a JSON object containing the key "books" and the generated list of book IDs as a value. Additionally, it returns the HTTP code for "OK" (200). The HTTP is specified as the second argument of the return command:

return {"books": books}, 200

Let us verify that the solution is correct by sending a GET request to the server endpoint */books* just like you did in the previous task. For testing, manually move one or more Gutenberg book files that you have saved in previous exercise into the books folder, so that it is no longer empty.

Expected result is something like the following. Notice that the browsers view source is used to see the JSON text.




**b. Creating an HTTP method to download a specific book**

Create an API GET method that will allow us to download the content of an existing book. The method receives the ID of the book as input, and returns the contents of the book as text.

Create a new Python method with the name download_book(book_id):

@app.route('/books/<book_id>', methods = ['GET'])
def download_book(book_id):

This method should be available on the server at the endpoint */books/book_id*, via the GET HTTP method. Where the book_id is the Gutenberg ID of the requested book.

Please write the code for the download_book(book_id) method. (1 point)

**Note**: You must verify the method argument should be an integer and also consider the case where the book is not existing on our book management system. (1 point)

c. **Creating an HTTP method for deleting books** (1 point)

Create a REST API DELETE method to be able to delete existing books.

create a new Flask Python method delete_book(book_id):

Windows example of calling HTTP DELETE method:

```
Invoke-RestMethod -Uri http://localhost:5000/books/12345  -Method 'Delete'
```

Linux  example of calling HTTP DELETE method:

```
curl -X "DELETE" http://localhost:5000/books/12345
```

If the method works properly, the file should be deleted and no longer shows up in the list of books.

**d. Creating an HTTP method for creating new books by downloading from Gutenberg**

Create a HTTP POST method to which the book ID can be sent, after which the server downloads the book corresponding to this ID from the Gutenberg repository and stores it in the books folder.

def add_book():

Example of calling the method through HTTP POST request in Windows:

```
Invoke-RestMethod -Uri "http://localhost:5000/books" -Method 'Post' -Body '{"book_id": "12345"}' -ContentType "application/json"
```

Example of calling the method through HTTP POST request in Linux:

```
curl -X POST -H "Content-Type: application/json" --data '{"book_id": "12345"}' http://localhost:5000/books
```

If the program works successfully, a new file should be created in the books folder. (2 points)

**Note:** For optimization, if the file already exists on the server, do not download it again. (1 point)

e. **Creating an HTTP method for searching words in a book** (2 points)

Create a new POST type HTTP API method to search for words in a book managed by the server. The HTTP method takes the book id and the word as an argument and returns a counter that shows how many times the specified word appeared in this book. This method should not re-download the book from Gutenberg, but should only search for the word in the book already on the server.

Create a method search_term_from_book:

This method should be available on the server at /book_search, via the POST HTTP method. The input should be a JSON data structure that contains the Gutenberg ID of the book and the search term. For example:

```
{"book_id": "224", "search_term": "blue"}
```

Example of calling the method in Windows:

```
Invoke-RestMethod -Uri "http://localhost:5000/book_search" -Method 'Post' -Body
'{"book_id": "224", "search_term": "blue"}' -ContentType "application/json"  | ConvertTo-Json
```

Example of calling the method in Linux:

```
curl -X POST -H "Content-Type: application/json" --data '{"book_id": "224", "search_term":
"blue"}'  http://localhost:5000/book_search
```

Expected result:

```
{
        "book_id":  "224",
        "occurrences":  31,
        "search_term":  "blue"
}
```

Follow these guidelines while submitting the exercises

1. Generate a PDF file with answers/screenshots with name being your studentid_EX6
2. Export the source code as a zip file
3. Put a pdf file with the answers for the questions
    a. For programming tasks show screenshots of the program output for the correct and wrong inputs
    b. For commands tried, provide the screenshots
4. The programs should guide the user in putting the correct input
    a. Have detailed comments of your logic
5. Do not copy code/answers from internet. If you copy, please leave a comment with the URL on that source code file
6. Plagiarism will not be tolerated and dealt through the formal channels of the university